

INT 453: Digital Multimedia

2024-2025

Lecturer : Dr. Maysa Khalil Noby
Dr. Reham Amin
Reham_Amin@ci.suez.edu.eg
Office: NG50

Lecture 5

Arithmetic Coding

books

- Fundamentals of Multimedia.
- Multimedia Technologies & applications for the 21st century.

Overview

This chapter covers:

- Arithmetic Coding
- Arithmetic Decoding
- Examples

Data Compression

Coding Recap

Coding rate is the average number of bits used to represent a symbol from a source.

For a given probability model, the entropy is the lowest rate at which the source can be coded.

In Huffman coding, when the alphabet size is large, the amount of deviation from the entropy is quite small, and vice versa.

Coding Recap

One solution for this problem is **blocking** in Huffman coding.

In which, it is more efficient to generate codewords for groups or sequences of symbols rather than to generate a separate codeword for each symbol in a sequence.

In order to find the Huffman coding for a sequence of length *m*, we need codewords for all possible sequences of length *m*.

This causes an exponential growth in the size of the code book.

Arithmetic Coding

- Arithmetic coding is a more modern coding method that usually outperforms Huffman coding in practice.
- The initial idea of arithmetic coding was introduced in Shannon's 1948 work.
- Various modern versions of arithmetic coding have been developed for newer multimedia standards: for example,
- Fast Binary Arithmetic Coding in JBIG,
- JBIG2 and JPEG-2000, and Context-Adaptive Binary Arithmetic Coding (CABAC) in H.264.



Arithmetic Coding

- Normally (in its non-extended mode), Huffman coding assigns each symbol a codeword that has an integral bit length.

- Number of bits for each input symbol. In practice, the input data is usually broken up into chunks to avoid error propagation.

Arithmetic coding

- Arithmetic coding is based on the concept of *interval subdividing*.
  - In arithmetic coding a source ensemble is represented by an interval between 0 and 1 on the real number line.
  - Each symbol of the ensemble narrows this interval.
  - As the interval becomes smaller, the number of bits needed to specify it grows.

Arithmetic coding (cont.,)

- It uses the probabilities of the source messages to successively narrow the interval used to represent the ensemble.
  - A high probability message narrows the interval less than a low probability message, so that high probability messages contribute fewer bits to the coded ensemble

Arithmetic coding

Assume we know the probabilities of each symbol of the data source,

- we can allocate to each symbol an interval with width proportional to its probability, and each of the intervals does not overlap with others.

- This can be done if we use the cumulative probabilities as the two ends of each interval. Therefore, the two ends of each symbol x amount to Q[x-1] and Q[x].

- Symbol x is said to own the range [Q[x-1], Q[x]).

Arithmetic Encoding

We begin with the interval [0,1] and subdivide the interval iteratively.

- For each symbol entered, the current interval is divided according to the probabilities of the alphabet.
- The interval corresponding to the symbol is picked as the interval to be further proceeded with.
- The procedure continues until all symbols in the message have been processed.

Arithmetic Encoding

- Since each symbol's interval does not overlap with others, for each possible message there is a unique interval assigned.
- We can represent the message with the interval's two ends [L,H). In fact, taking any single value in the interval as the encoded code is enough, and usually the left end L is selected.

How to compute each point in the new intervals?

- The first line contains the commulative probabilities
- At each iteration (new line) after the first line,
  - 1- compute the difference (d) between the current Lower and current upper:

$$d\ (diff) = upper - lower$$

  - 2- compute each probability in the new interval by
    - Probability(the preceding lower in the same line) +d(diff)*probability(current symbol)

Arithematic Encoding Algorithm

Generating Codeword for Encoder

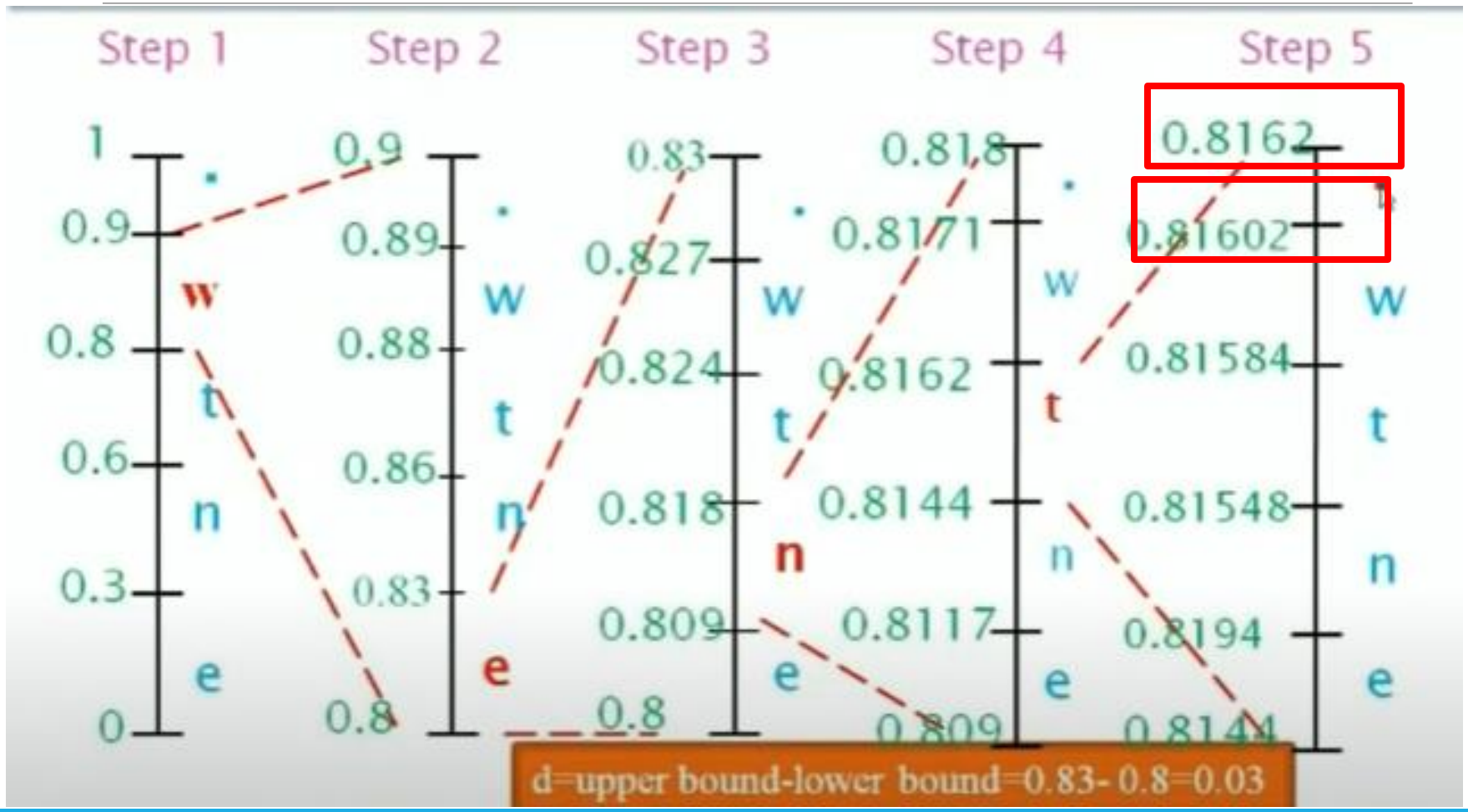
```
BEGIN
    code = 0;
    k = 1;
    while (value(code) < low)
        {
          assign 1 to the kth binary fraction bit;
          if (value(code) > high)
             replace the kth bit by 0;
          k = k + 1;
        }
END
```

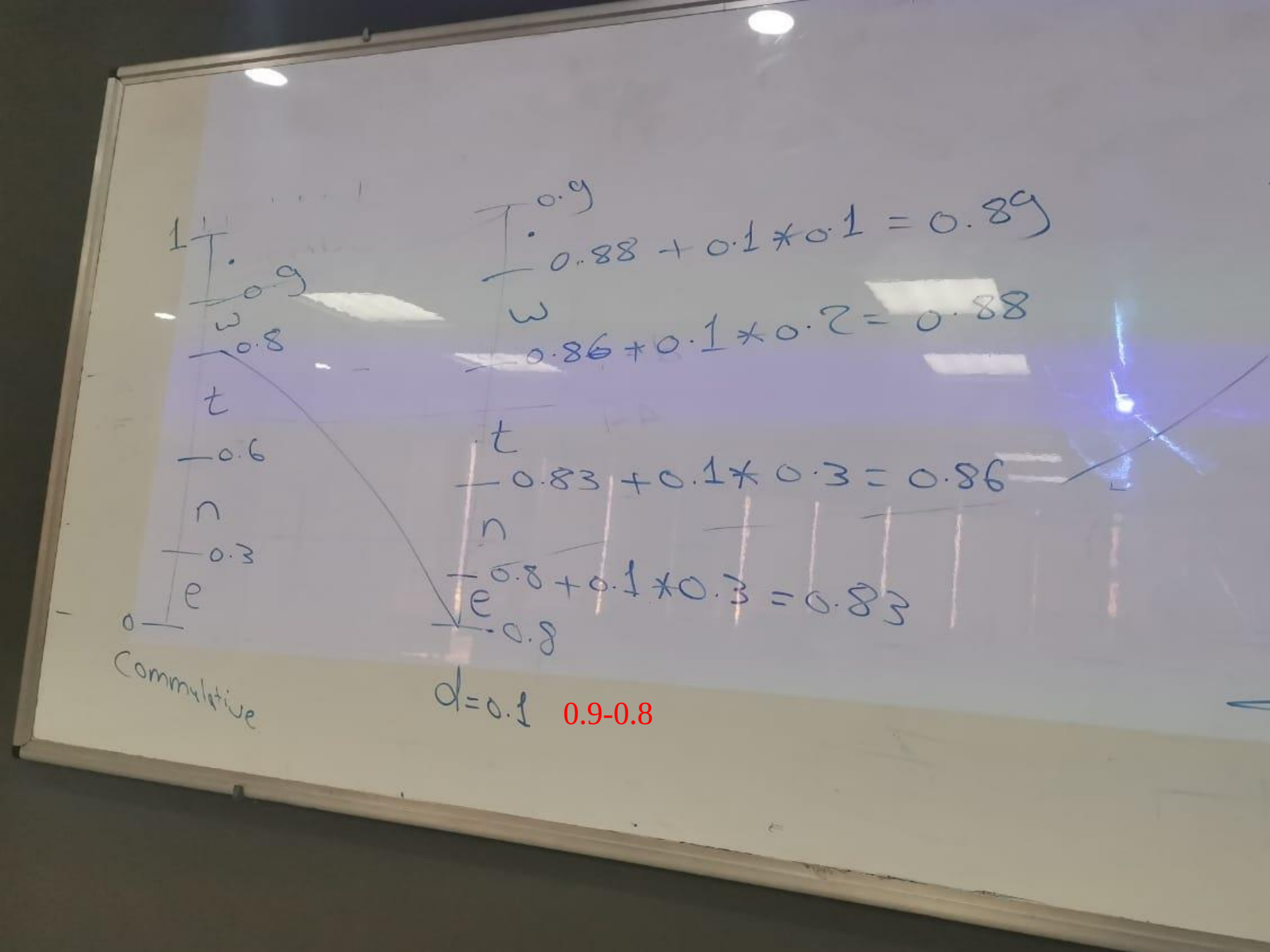
Example 1

Consider the transmission of a message “went.” compromising a string of characters with probability:

e →0.3, n →0.3, T →0.2, W →0.1, ‘.’→0.1.

Example 1 (cont.,)




1
T.
0.9
w
0.8
t
0.6
n
0.3
e
0
Commulative
T.
0.9
0.88 + 0.1 * 0.1 = 0.89
w
0.86 * 0.1 * 0.2 = 0.88
t
0.83 + 0.1 * 0.3 = 0.86
n
0.8 + 0.1 * 0.3 = 0.83
e
0.8
d=0.1
0.9-0.8

Solution

Answer:

The Arithmetic code word is (between the 2 last upper intervals)

0.81602 < Code word < 0.8162

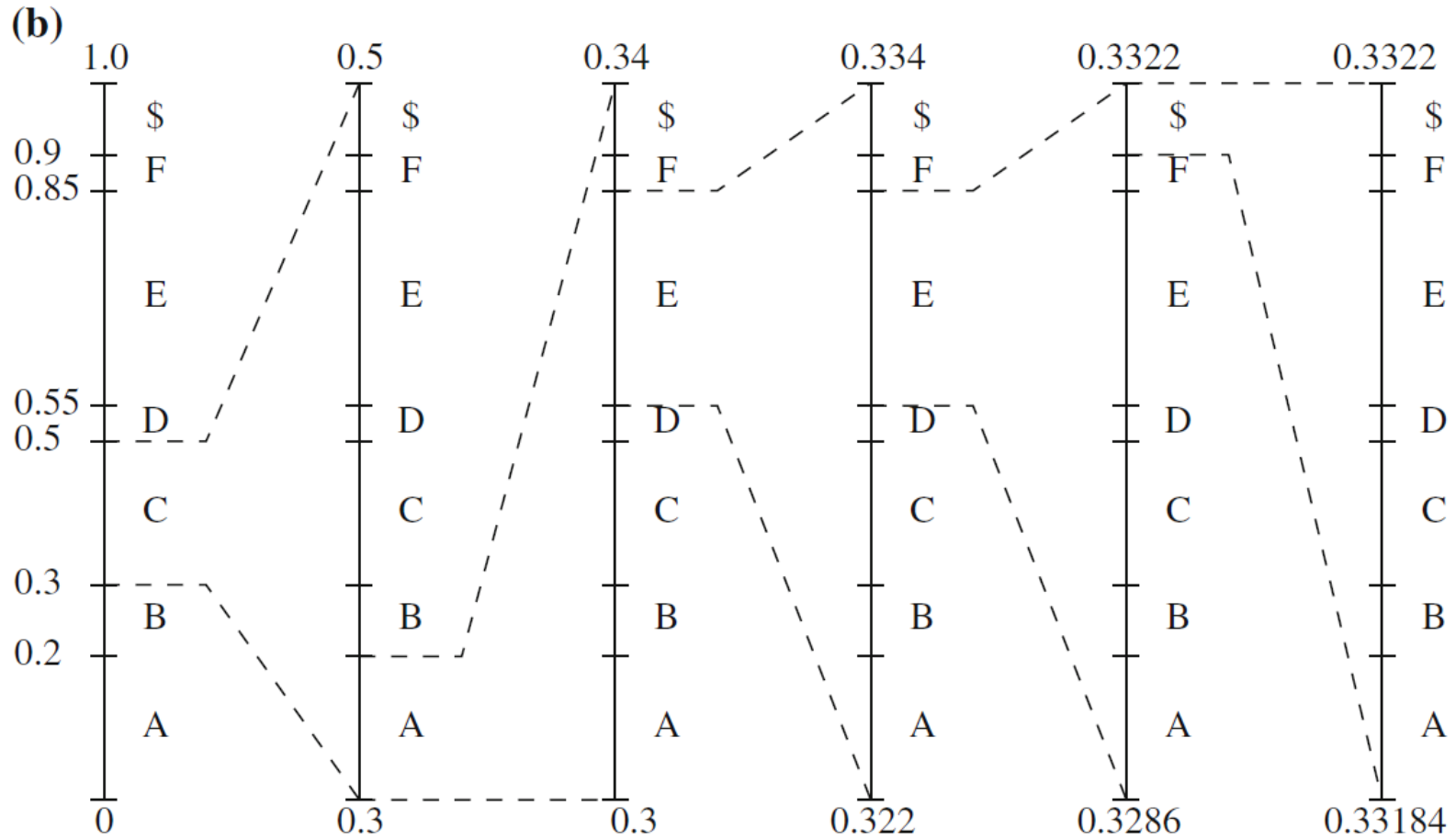
Encoding Example
Ex. 2 (home work)

Encode the String **: CAEE$**

Using the following table

Symbol	Probability	Range	Range_low	Range_high
A	0.2	[0,0.2)	0	0.2
B	0.1	[0.2,0.3)	0.2	0.3
C	0.2	[0.3,0.5)	0.3	0.5
D	0.05	[0.5,0.55)	0.5	0.55
E	0.3	[0.55,0.85)	0.55	0.85
F	0.05	[0.85,0.9)	0.85	0.9
$	0.1	[0.9,1.0)	0.9	1.0

Encoding Example
Ex. 2



Encoding Example
Ex. 2

- The encoding process is illustrated in Fig. 7.9b and c, in which a string of symbols **CAEE$** is encoded. Initially, low = 0, high = 1.0, and range = 1.0.
- The first symbol is C, Range_low(C) = 0.3, Range_high(C) = 0.5, so after the symbol C,
- low = 0 + 1.0 × 0.3 = 0.3, high = 0 + 1.0 × 0.5 = 0.5. The new range is now reduced to 0.2.
- For clarity of illustration, the ever-shrinking ranges are enlarged in each step (indicated by dashed lines)

Encoding Example
Ex. 3

- After the second symbol A, low, high, and range are 0.30, 0.34, and 0.04. The process repeats itself until after the terminating.
- symbol $ is received. By then low and high are 0.33184 and 0.33220, respectively.
- It is apparent that finally we have
- range = PC × PA × PE × PE × P$ = 0.2 × 0.2 × 0.3 × 0.3 × 0.1 = 0.00036

Encoding Example
Ex. 3

Symbol	low	high	range
	0	1.0	1.0
C	0.3	0.5	0.2
A	0.30	0.34	0.04
E	0.322	0.334	0.012
E	0.3286	0.3322	0.0036
$	0.33184	0.33220	0.00036

- The final step in Arithmetic encoding calls for the generation of a number that falls within the range [*low*, *high*). The above algorithm will ensure that the shortest binary codeword is found.

Arithematic Decoding Algorithm

```
BEGIN
   get binary code and convert to decimal value = value(code);
   Do
        {
          find a symbol s so that
               Range_low(s) <= value < Range_high(s);
          output s;
          low = Rang_low(s);
          high = Range_high(s);
          range = high - low;
          value = [value - low] / range;
        }
   Until symbol s is a terminator
END
```

Decoding Example 1

Arithmetic decoding example

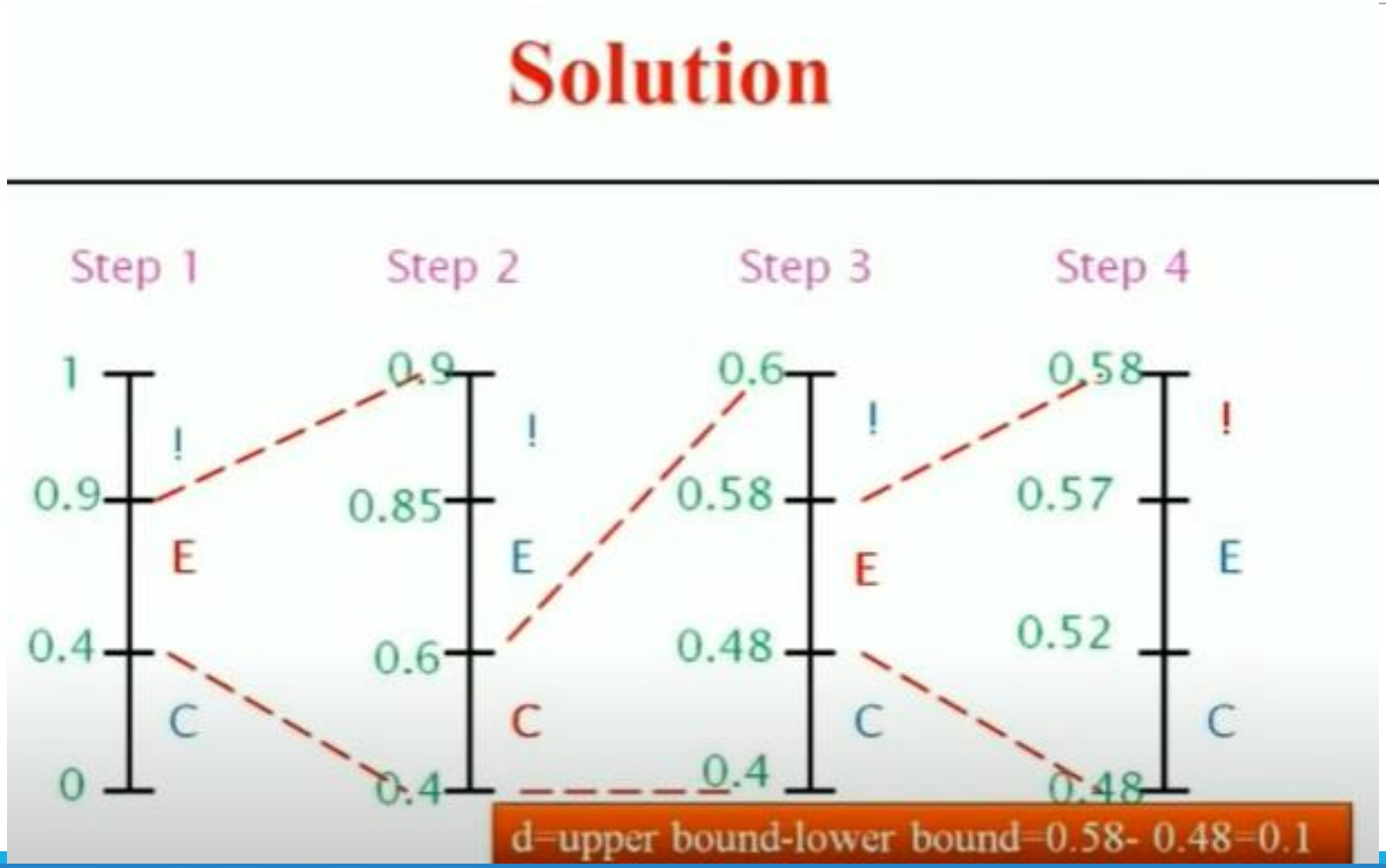
Decode the message 0.572 given the coding model

Symbol	!	C	E
Probability	0.1	0.4	0.5

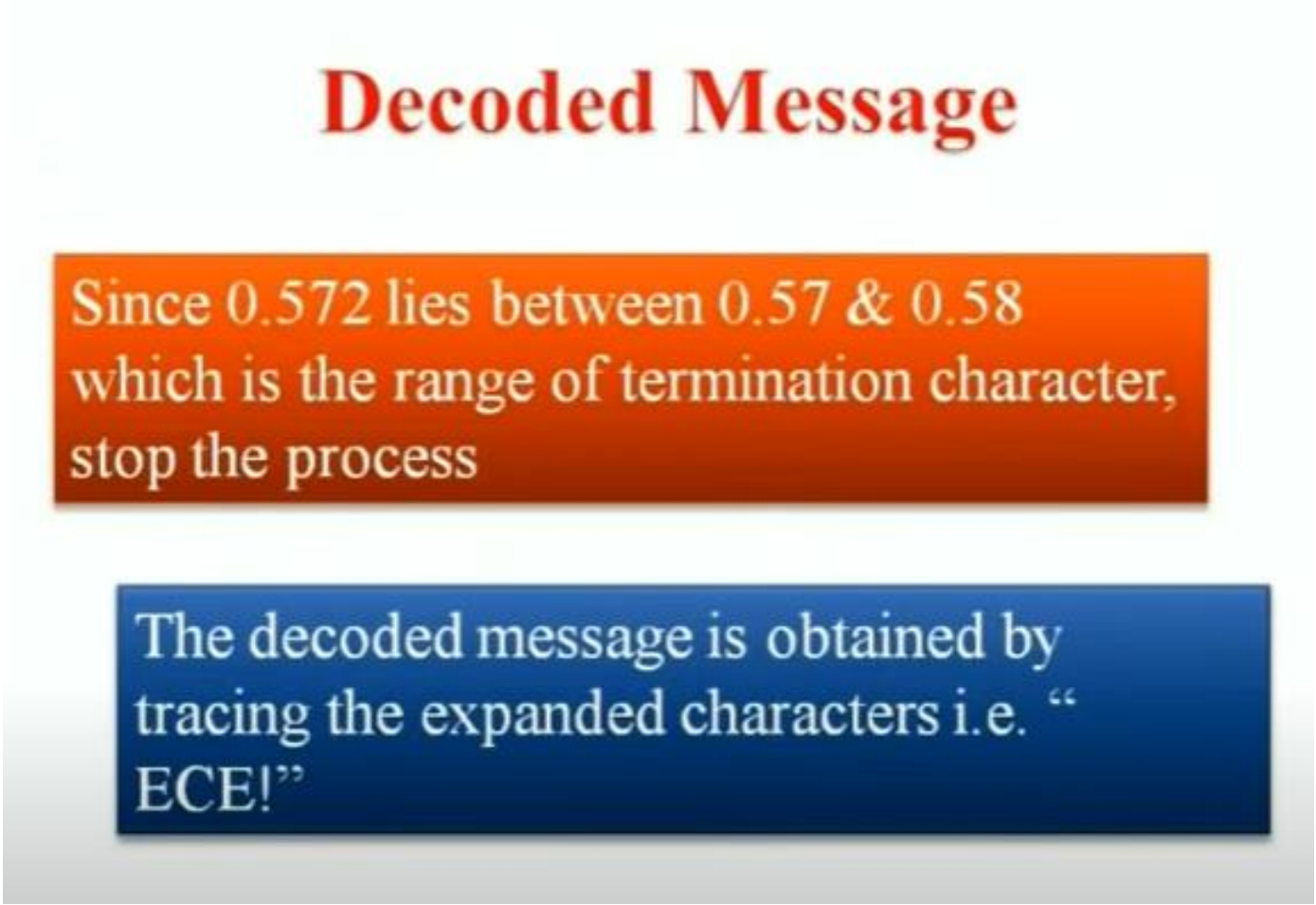
Symbol	!	C	E
Probability	0.1	0.4	0.5

Symbol	Probability	Range	Range_low	Range_high
A	0.2	[0,0.2)	0	0.2
B	0.1	[0.2,0.3)	0.2	0.3
C	0.2	[0.3,0.5)	0.3	0.5
D	0.05	[0.5,0.55)	0.5	0.55
E	0.3	[0.55,0.85)	0.55	0.85
F	0.05	[0.85,0.9)	0.85	0.9
$	0.1	[0.9,1.0)	0.9	1.0

Decoding Example 1 (cont.,)



Decoding Example (cont.,)



Arithmetic Decoding Ex. 2

Arithmetic decoding example

Decode the message 0.33203125 given the coding model

Symbol	Probability	Range	Range_low	Range_high
A	0.2	[0,0.2)	0	0.2
B	0.1	[0.2,0.3)	0.2	0.3
C	0.2	[0.3,0.5)	0.3	0.5
D	0.05	[0.5,0.55)	0.5	0.55
E	0.3	[0.55,0.85)	0.55	0.85
F	0.05	[0.85,0.9)	0.85	0.9
$	0.1	[0.9,1.0)	0.9	1.0

Arithmetic Decoding Ex. 2

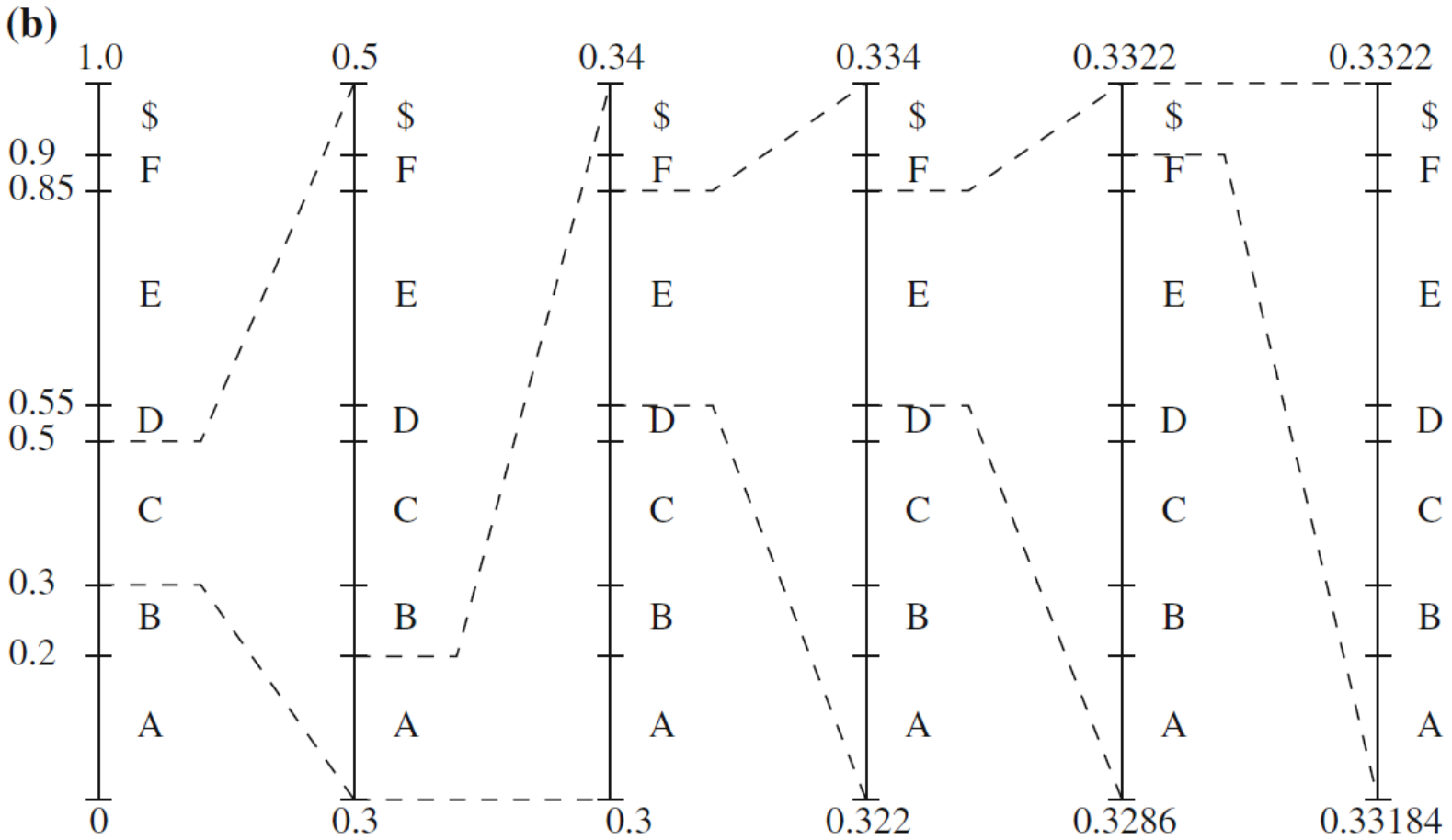
The decoding process for the above example. Initially, value = 0.33203125.

Since Range_low*(C)* = 0.3 ≡ 0.33203125 < 0.5 = Range_high*(C)*,

The first output symbol is C. This yields *value* = [0.33203125 − 0.3]/0.2 = 0.16015625, which in turn determines that the second symbol is A.

Eventually, *value* is 0.953125, which falls in the range [0.9, 1.0*)* of the terminator $.

Arithmetic Decoding Ex. 2



Review Questions

Generate the code word for the string “BCAB” with the probability A →0.4, B →0.3, C →0.2, D →0.1.

Encode the String**: CAEE$** Using the following table.

Symbol	Probability
A	0.2
B	0.1
C	0.2
D	0.05
E	0.3
F	0.05
$	0.1

Review Questions

Decode the message 0.752 given the coding model

Symbol	m	C	A	E
Probability	0.1	0.3	0.2	0.4

Decode the message 0.624 given the coding model

Symbol	S	B	A	E
Probability	0.1	0.2	0.3	0.4

Assignment

- What are the different types of Arithmetic Coding?
- What are the limitations of using Arithmetic Coding?
- How to find the compression ratio using Arithmetic Coding.

Questions

Thank You